arXiv:2310.10348v2 [cs.LG] 20 Nov 2023

Attribution Patching Outperforms Automated Circuit Discovery

Aaquib Syed
University of Maryland, College Park
asyed04@umd.edu

Can Rager
Independent
canrager@gmail.com

Arthur Conmy
Independent
arthurconmy@gmail.com

Abstract

Automated interpretability research has recently attracted attention as a potential research direction that could scale explanations of neural network behavior to large models. Existing automated circuit discovery work applies activation patching to identify subnetworks responsible for solving specific tasks (circuits). In this work, we show that a simple method based on attribution patching outperforms all existing methods while requiring just two forward passes and a backward pass. We apply a linear approximation to activation patching to estimate the importance of each edge in the computational subgraph. Using this approximation, we prune the least important edges of the network. We survey the performance and limitations of this method, finding that averaged over all tasks our method has greater AUC from circuit recovery than other methods.[1]

1 Introduction

Mechanistic interpretability is a subfield of AI interpretability that focuses on attributing model behaviors to its components, thus reverse engineering the network [1]. This field aims to identify subnetworks (circuits) within the model which are responsible for solving specific tasks [2]. Prior attempts at finding circuits in language models have led to finding networks of attention heads and multi-layer perceptrons (MLPs) that partially or fully explain model behaviors at tasks such as indirect object identification, modular arithmetic, completion of docstrings, and predicting successive dates [3, 4, 5, 6]. However, almost all previous work has been limited to relatively small models since manually applying mechanistic interpretability methods has not currently scaled to end-to-end circuits in larger models [7].

It may be important to scale interpretability to large models as these are the neural networks most widely deployed and used by a wide range of people. Currently, we have little understanding into these models work and failure modes are not always found ahead of deployment. If successful, scaled interpretability could address a wide variety of concerns about the lack of transparency of language models [8], in addition to speculative risks about the alignment of machine learning systems [9].

Automated Circuit Discovery (ACDC; [10]) attempts to automate a large portion of the mechanistic interpretability workflow — the pruning of edges between attention heads and MLPs that do not affect the task being studied. ACDC begins with a computational graph, and recursively calculates the importance of an edge in the graph for a specific task. In our work, we use edges to refer to activations inside models between two components (Section 2 describes this motivation further). ACDC's pruning algorithm applies **activation patching**. (Note that **activation patching** is not **attribution patching**. Both are defined in full in Section 3.3.) At a high level, activation patching edits a specific activation in a model forward pass and measures a model statistic (e.g loss) under this

[1] Our code is available at https://github.com/Aaquib111/acdcpp

37th Conference on Neural Information Processing Systems (NeurIPS 2023) ATTRIB Workshop.

intervention. Activation patching is inefficient for circuit discovery because getting each statistic about model activations requires another forward pass. Our work uses **attribution patching** to recover circuits more efficiently (Section 3.3).

Our main contributions are:

1. Introducing a method for using attribution patching on all computational graph edges for automated circuit discovery (Edge Attribution Patching, Section 3.3).
2. Benchmarking Edge Attribution Patching vs existing circuit discovery methods (Section 4).
3. Finding and explaining some limitations with Edge Attribution Patching (Section 5).

2 Related Work

Automated Circuit Discovery refers to finding the important subgraph of models' computational graphs for performance on particular tasks [10]. Existing algorithms include efficient heuristics [11] and gradient-descent based methods [12, 13]. ACDC is related to pruning [14] and other compression techniques [15], but differs in how the compressed networks are reflective of the circuits that model uses to compute outputs to certain tasks and the goal of ACDC is not to speed up forward passes (all techniques studied in this work slow forward passes).

Activation Patching is a technique for analyzing the role of individual components in a model. It involves targeted manipulations of activations during a forward pass (further explained in Section 3.1). Previous works applied this technique under various names, such as Interchange Interventions [16], Causal Mediation Analysis [8] and Causal Tracing [17]. We adapt the terminology used by Conmy et al. [10].

Transformer Circuits. Our work builds upon the framework for understanding transformers for interpretability as introduced by Elhage et al. [18]. The important details include how they formulate forward passes of transformer models. Individual attention heads and MLPs (collectively called nodes) read and write information to a central communication channel, also called the residual stream. In these terms we can examine dependencies of nodes with the output of earlier nodes, i.e we can measure the effect of attention heads in layer 0 on the attention heads in layer 2. In the following, we view these dependencies as edges between nodes, building on existing work using this perspective [5, 6, 3].

3 Edge Attribution Patching

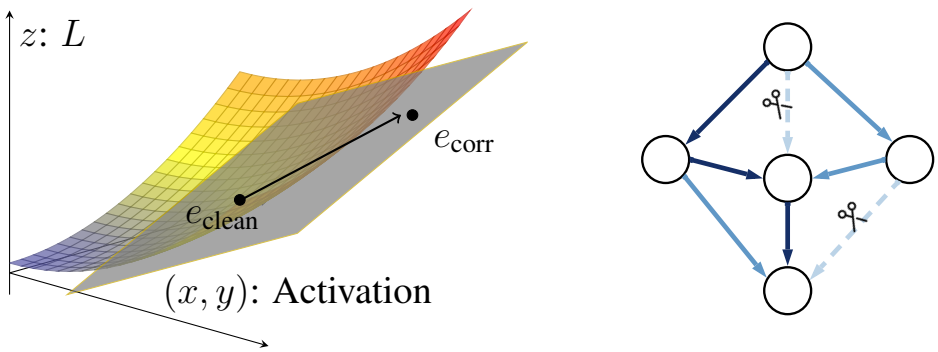
We present **Edge Attribution Patching** (EAP) as a technique to identify relevant model components for solving a specific task. In the following, we view language models as directed, acyclic graphs. In these terms, we aim to find small subgraphs that retain good performance on narrow tasks. We determine the importance of a specific edge through targeted manipulation of activations during a forward pass. We compare two approaches, Attribution Patching and Activation Patching, in order to motivate EAP.

3.1 Activation Patching

Activation patching refers to replacing the activations from one model forward pass with the activations from a different forward pass. This method is typically applied to measure the counterfactual importance of model components, i.e. to measure a statistic $L(x)$ from model outputs under the activation patching, where x is an input prompt. For example, L often represents loss or logit difference [3].

Following existing work (Section 2), we study the effect of activation patching on specific model edges by setting these equal to activations from different forward passes. Concretely, suppose that an edge E in the computational graph has activation e_{corr} on some corrupted prompt. In this work, we use the change in metric under activation patching

$$|L(x_{\text{clean}}|\ \text{do}(E = e_{\text{corr}})) - L(x_{\text{clean}})| \tag{1}$$



(a) Attribution Patching (Section 3.3) approximates the difference in metric L caused by corrupting edges.

(b) Removing the least important edges.

Figure 1: Edge Attribution Patching (EAP)

to measure the impact of edge E. We use do-notation from causality [19] to emphasise that activation patching is a causal intervention.

3.2 Attribution Patching

Activation patching slows ACDC since each measurement (like Equation (1)) requires another forward pass. ***Attribution patching*** [20] is a technique for estimating Equation (1) for many different edges E using only two forward passes and one backward pass.[2] It linearly approximates the metric difference after corrupting a single edge in the computational graph (Figure 1) by expanding L as a function of the edge activation as a Taylor series with terms up to the first order:

$$L(x_{\text{clean}}|\,\text{do}(E = e_{\text{corr}})) \approx L(x_{\text{clean}}) + \underbrace{(e_{\text{corr}} - e_{\text{clean}})^{\top} \frac{\partial}{\partial e_{\text{clean}}} L(x_{\text{clean}}|\,\text{do}(E = e_{\text{clean}}))}_{\text{Call this } \Delta_e L\text{, the } \textbf{attribution score}.} . \tag{2}$$

A simple rearrangement implies that Equation (1) is approximately equal to $|\Delta_e L|$ (3) which we call the **absolute attribution score** for the rest of this paper. In this work we always compute this score across a set of $(x_{\text{clean}}, x_{\text{corr}})$ pairs and take the mean.

In practice, all gradients needed to calculate the attribution scores come from intermediate terms computed in one ordinary backwards pass[3] in PyTorch [21], hence attribution patching is extremely efficient.

3.3 Edge Attribution Patching

We can use the insights from Section 3.2 to build an automated circuit discovery algorithm. This takes two steps: i) use Equation (2) to obtain absolute attribution scores for the importance of all edges in the computational graph and then ii) sort these scores and keep the top k edges in a circuit. We use **Edge Attribution Patching** (EAP) to refer to this algorithm. In the rest of the work we report results for all k values when we evaluate EAP (similar to HISP in [10]).

Note that one limitation of attribution patching is that it will not work when the gradient of the metric is the zero vector. Conmy et al. [10] recommended the use of KL divergence as a metric, which is i) equal to 0 when we run the model without ablations and ii) a non-negative metric. Therefore the zero point is a global minimum and hence all gradients are the zero vector at this point. In this work we use the 'task-specific metrics' (not KL divergence) from [10] so avoid this issue.

[2] Attribution patching (like activation patching) also applies to nodes and other model internal components that aren't edges, but we only use edges in this work.

[3] In Appendix F we show how only one backwards pass is required.